

APLIKASI ABSENSI SISWA

Oleh:

- 1. Muhammad Nurullah Irfan Bachtiar [103032500067]**
- 2. William [103032500125]**

Mata Kuliah: Algoritma Pemrograman

Kelas: IT-49-01

Kode Dosen: MIU



**PROGRAM STUDI TEKNOLOGI INFORMASI
TELKOM UNIVERSITY**

2026

BAB I PENDAHULUAN

1.1. Latar Belakang

Sistem manajemen informasi akademik pada tingkat pendidikan tinggi memerlukan tingkat akurasi dan efisiensi yang tinggi dalam pengelolaan data transaksional harian. Salah satu pilar penunjang kedisiplinan dan objektivitas evaluasi hasil belajar mahasiswa adalah pencatatan kehadiran atau presensi kelas. Pengelolaan presensi secara konvensional, seperti pencatatan manual pada lembar kertas atau penggunaan berkas spreadsheet terpisah, sering kali menghadapi kendala degradasi fisik berkas, risiko manipulasi identitas, ketidaksinkronan data, serta besarnya waktu yang terbuang untuk rekapitulasi data. Hambatan-hambatan administratif tersebut mendorong urgensi perancangan sistem informasi presensi yang terintegrasi, andal, dan mampu beroperasi secara deterministik pada memori komputer.

Dalam konteks akademik di Telkom University, pengembangan aplikasi ini juga berfungsi sebagai sarana implementasi praktis atas konsep-konsep fundamental dan lanjutan dari mata kuliah Algoritma Pemrograman. Struktur bahasa pemrograman Go yang bertipe statis (*statically typed*) dan memiliki performa tinggi sangat ideal untuk melatih kedisiplinan perancangan alokasi memori melalui larik statis (*static array*), pemodelan objek dunia nyata menggunakan tipe data bentukan (*struct*), serta optimalisasi manipulasi state memori melalui mekanisme pointer. Melalui integrasi algoritma pencarian biner (*binary search*) dan pengurutan (*sorting*) berbasis perbandingan numerik presisi tinggi, aplikasi ini dirancang untuk mengatasi kerawanan duplikasi data dan inefisiensi pengolahan laporan berkala.

1.2. Deskripsi Singkat Program

Aplikasi Absensi Siswa merupakan sebuah perangkat lunak berbasis Command Line Interface (CLI) yang dikembangkan menggunakan bahasa pemrograman Go. Program ini mengandalkan pengelolaan penyimpanan langsung di dalam memori utama (*in-memory storage*) untuk mensimulasikan database transaksional relasional secara mandiri tanpa ketergantungan pada middleware eksternal. Tujuan utama dari program ini adalah menyediakan platform terpadu bagi administrator kelas untuk memelihara profil mahasiswa, mencatat log kehadiran harian secara kronologis, dan mengekstrak metrik statistik keaktifan belajar secara instan.

Data utama yang dikelola di dalam program diklasifikasikan ke dalam dua kategori besar, yaitu data master mahasiswa (terdiri atas Nomor Induk Siswa dan Nama) dan data log kehadiran (mencakup tanggal sesi, referensi identitas, dan status presensi). Seluruh data tersebut disimpan dalam struktur array statis dengan batas kapasitas terdefinisi guna menjamin keamanan memori dari ancaman luapan data (*overflow*). Aksesibilitas program dikendalikan secara modular melalui tiga menu operasional utama, yaitu Menu Mahasiswa untuk manajemen data profil, Menu Presensi

untuk pencatatan transaksi kelas harian, dan Menu Statistik untuk analisis agregat performa kehadiran mahasiswa.

BAB II PERANCANGAN

2.1. Deskripsi Data

Perancangan struktur data dalam program ini menerapkan prinsip modularitas objek, di mana setiap konsep dunia nyata didefinisikan ke dalam bentuk tipe data bentukan (*struct*). Untuk menjamin konsistensi relasi antar-entitas, program menggunakan Nomor Induk Siswa (NIS) sebagai pengidentifikasi unik (*primary key*) yang menghubungkan entitas mahasiswa dengan log kehadiran mereka.

Terdapat empat tipe data bentukan (*struct*) utama yang saling berelasi di dalam aplikasi ini:

1. **Dates:** Struktur pembantu yang mengkapsulasi informasi penanggalan. Penggunaan atribut terpisah untuk hari, bulan, dan tahun bertujuan mempermudah operasi pengurutan kronologis serta proses validasi batas nilai penanggalan (misalnya batasan hari 1-31 dan bulan 1-12).
2. **Student:** Struktur data master yang menampung identitas fisik setiap mahasiswa terdaftar. Atribut *NIS* bertipe string digunakan untuk mengakomodasi nomor induk dengan digit panjang tanpa risiko kegagalan representasi angka besar, sedangkan atribut *Name* menyimpan nama lengkap mahasiswa.
3. **Attendance:** Struktur transaksional yang merepresentasikan log kehadiran. Atribut *Date* mengintegrasikan struct *Dates*, sedangkan atribut *NIS* dan *Name* memetakan langsung ke data mahasiswa. Penyimpanan nama mahasiswa secara langsung di dalam log kehadiran (*denormalization*) diterapkan untuk mengoptimalkan performa pembacaan dan penyajian laporan pada terminal tanpa memicu overhead operasi pencarian berulang pada array mahasiswa. Atribut *Presence* memegang status boolean presensi.
4. **Statistics:** Struktur analisis yang dibentuk secara dinamis untuk menyajikan data pelaporan agregat. Atribut *TotalPresence* dan *TotalMeetings* menyimpan parameter frekuensi kehadiran dan total sesi, yang kemudian dikalkulasi secara matematis menghasilkan atribut *Percentage* berpresisi tinggi.

Alokasi penyimpanan fisik pada memori ditangani oleh tiga tipe array statis kustom, yaitu *TabStudent* yang dibatasi oleh konstanta *maxStudent* = 50, *TabAttendance* yang dibatasi oleh konstanta *maxAttendance* = 18250, serta *TabStatistics*. Batasan maksimal 18.250 pada data kehadiran dirancang secara sistematis untuk mengakomodasi pencatatan kehadiran harian maksimal 50 mahasiswa selama satu tahun penuh ($365 \text{ hari} \times 50 \text{ mahasiswa} = 18.250 \text{ record}$).

Rincian lengkap dari rancangan atribut data, tipe data, serta fungsi masing-masing elemen disajikan pada Tabel 1:

Nama Struct	Atribut	Tipe Data	Deskripsi Operasional
Dates	<i>Day</i>	<i>int</i>	Menyimpan nilai hari dalam format angka (1 s.d. 31)
	<i>Month</i>	<i>int</i>	Menyimpan nilai bulan dalam format angka (1 s.d. 12)
	<i>Year</i>	<i>int</i>	Menyimpan nilai tahun pelaksanaan kelas (contoh: 2026)
Student	<i>NIS</i>	<i>string</i>	Nomor Induk Siswa sebagai pengenalan unik (hanya karakter numerik)
	<i>Name</i>	<i>string</i>	Nama lengkap mahasiswa
Attendance	<i>Date</i>	<i>Dates</i>	Tanggal pelaksanaan presensi harian
	<i>NIS</i>	<i>string</i>	Kunci relasi yang merujuk pada NIS mahasiswa bersangkutan

	Name	string	Nama lengkap mahasiswa yang disalin saat pencatatan presensi
	Presence	bool	Flag kehadiran (true untuk Hadir, false untuk Absen/Alfa)
Statistics	NIS	string	Kode pengenal mahasiswa yang dievaluasi
	Name	string	Nama lengkap mahasiswa yang dievaluasi
	TotalPresence	int	Akumulasi sesi kelas dengan status Presence = true
	TotalMeetings	int	Total sesi pertemuan kelas yang pernah diselenggarakan
	Percentage	float64	Persentase rasio kehadiran mahasiswa

```
const (
    maxStudent = 50
```

```

    maxAttendance = 18250
)

type Student struct {
    NIS    string
    Name   string
}

type Dates struct {
    Day    int
    Month  int
    Year    int
}

type Attendance struct {
    Date    Dates
    NIS      string
    Name     string
    Presence bool
}

type Statistics struct {
    NIS           string
    Name          string
    TotalPresence int
    TotalMeetings int
    Percentage     float64
}

type TabStudent [maxStudent]Student
type TabAttendance [maxAttendance]Attendance
type TabStatistics [maxStudent]Statistics

```

2.2. Daftar Fungsionalitas Program

Program ini mengusung pendekatan fungsional terstruktur dengan membagi sistem ke dalam beberapa modul independen yang saling berinteraksi melalui passing parameter. Fungsionalitas utama aplikasi dirancang untuk mendukung siklus hidup data akademik secara utuh.

Seluruh fungsionalitas program beserta kegunaan dan aspek validasi internalnya dipetakan secara komprehensif pada Tabel 2:

Menu Utama	Fungsionalitas Operasional	Alur Proses & Aturan Validasi (Constraint)
Sistem Utama	Inisialisasi Data Dummy (<i>Seeding</i>)	Mengisi memori dengan 10 record mahasiswa awal dan 30 record kehadiran historis yang tersebar pada tanggal 1, 2, dan 3 Januari 2026. Proses ini memastikan sistem langsung siap diuji tanpa input manual.
Menu Mahasiswa	Registrasi Mahasiswa Baru (<i>Add</i>)	Menginput NIS dan Nama. Menolak input jika NIS mengandung karakter non-numerik, mendeteksi duplikasi NIS melalui <i>Binary Search</i> secara instan, dan memicu <i>Auto-Sorting</i> untuk menjaga urutan menaik array.
	Pembaruan Profil (<i>Edit</i>)	Memilih indeks mahasiswa terdaftar untuk diperbarui namanya. Melakukan proses penyalinan kaskade (<i>cascade update</i>) untuk menyalin nama baru tersebut ke seluruh transaksi kehadiran historis di <i>attendanceData</i> .

	Penghapusan Data (<i>Delete</i>)	Menghapus record mahasiswa berdasarkan indeks. Menggeser semua indeks array di sebelah kanan elemen terhapus ke kiri agar data tetap kontigu. Menghapus data presensi terkait secara kaskade.
	Pencarian Cepat (<i>Search</i>)	Mencari mahasiswa secara presisi berdasarkan NIS menggunakan pencarian biner atau pencarian nama dengan pencarian sekuensial.
	Pengurutan Data (<i>Sort</i>)	Menyusun ulang tampilan data mahasiswa berdasarkan NIS atau Nama secara menaik (<i>ascending</i>) atau menurun (<i>descending</i>).
	Visualisasi Data Master (<i>Show</i>)	Menampilkan representasi tabel dari seluruh mahasiswa aktif yang terdaftar di dalam array <i>studentData</i> .
Menu Presensi	Pencatatan Presensi Baru	Meminta input tanggal baru. Melakukan iterasi ke seluruh daftar mahasiswa aktif dan

		meminta konfirmasi kehadiran satu per satu secara interaktif.
Menu Statistik	Analisis Rekapitulasi Kelas	Menghitung total kehadiran per mahasiswa dibandingkan dengan jumlah pertemuan kelas yang ada. Menyajikan persentase kehadiran dan mengurutkannya secara menurun untuk memantau mahasiswa dengan keaktifan terendah.

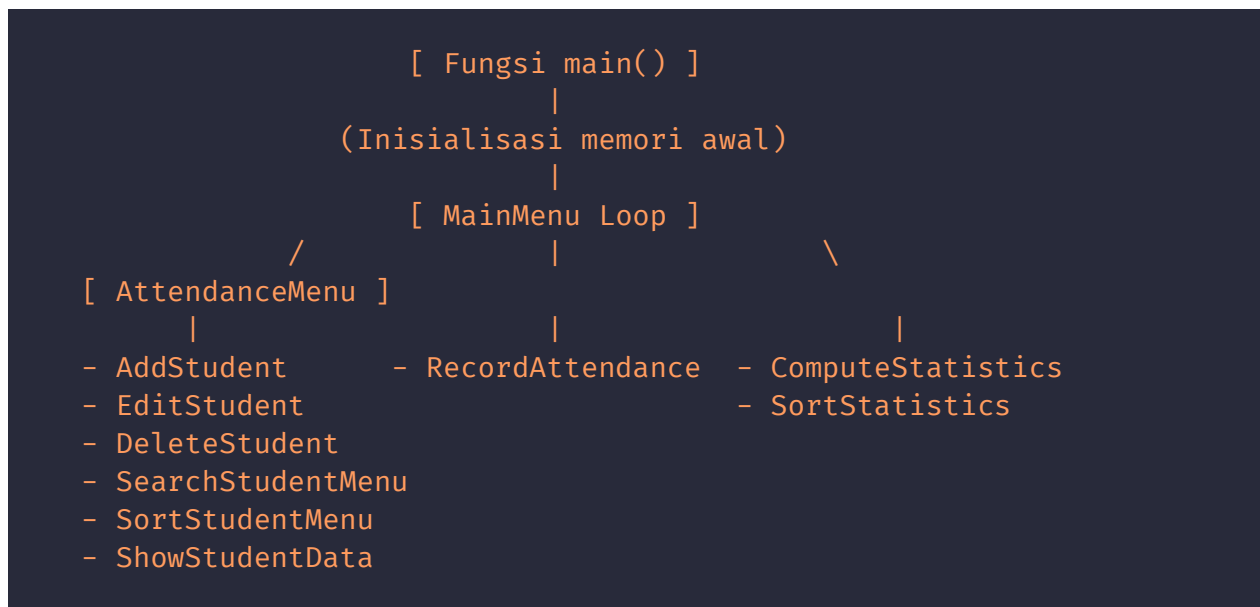
BAB III IMPLEMENTASI

3.1. Struktur Program

Program ini diimplementasikan menggunakan arsitektur pemrograman prosedural terstruktur dalam bahasa pemrograman Go. Penyimpanan status aplikasi (*state management*) selama runtime dikelola melalui dua variabel global utama yang dialokasikan di dalam memori heap komputer, yaitu *studentData* bertipe *TabStudent* dan *attendanceData* bertipe *TabAttendance*.

Akurasi pelacakan jumlah data riil yang tersimpan di dalam array statis dikelola melalui pointer variabel integer, yaitu *nS* (*number of students*) dan *nA* (*number of attendances*). Penggunaan pointer **int* ini sangat krusial; setiap kali subprogram menambah atau menghapus elemen array, nilai variabel di fungsi pemanggil (*main*) ikut berubah secara sinkron. Hal ini menghindari ketidaksinkronan data dan mencegah terjadinya kesalahan indeks di luar batas (*index out of bounds*).

Hubungan relasional antar-subprogram diatur melalui sebuah hierarki eksekusi satu arah. Alur eksekusi aplikasi secara sistematis digambarkan pada skema relasi subprogram berikut:



Di dalam *StudentMenu*, fungsionalitas pengurutan internal secara dinamis dipanggil oleh fungsi modifikasi data untuk menjamin integritas urutan. Sebagai contoh, prosedur *AddStudent* akan memanggil *SortStudentAsc* setelah record berhasil ditambahkan, sehingga status array *studentData* kembali teratur sempurna dan siap diakses oleh algoritma *BinarySearchStudent* pada operasi registrasi berikutnya.

3.2. Daftar Subprogram

Setiap fungsi dan prosedur dalam kode program *main.go* dirancang secara modular dengan tanggung jawab tunggal (*single responsibility principle*). Struktur parameter masukan dirancang secara selektif untuk mengoptimalkan efisiensi memori dengan menghindari penyalinan objek berukuran besar (*pass by value*).

Daftar subprogram esensial yang membangun sistem ini disajikan secara mendalam pada Tabel 3:

Nama Subprogram	Struktur Deklarasi / Parameter	Perilaku Logis & Hasil yang Diharapkan
<i>main</i>	<i>func main()</i>	<i>Entry point</i> aplikasi. Menginisialisasi <i>nStudent</i> = 0 dan <i>nAttendance</i> = 0, mengisi data dummy, lalu mentransfer kontrol eksekusi ke loop <i>MainMenu</i> .
<i>MainMenu</i>	<i>func MainMenu(scanner *bufio.Scanner, nS *int, nA *int)</i>	Mengelola interaksi CLI tingkat atas. Menggunakan scanner untuk menangkap input pilihan menu dan mengalihkan alur program ke sub-menu yang sesuai.
<i>StudentMenu</i>	<i>func StudentMenu(scanner *bufio.Scanner, nS *int, nA *int) bool</i>	Menyajikan pilihan operasi manajemen mahasiswa. Mengembalikan nilai boolean <i>true</i> untuk menghentikan program atau <i>false</i> untuk kembali

		ke tingkat menu sebelumnya.
SeedDummyData	func SeedDummyData(nS *int, nA *int)	Memasukkan 10 record profil mahasiswa dan 30 log kehadiran ke dalam array global secara langsung. Mengatur status awal *nS = 10 dan *nA = 30 .
AddStudent	func AddStudent(scanner *bufio.Scanner, nS *int)	Mendaftarkan mahasiswa baru jika kapasitas belum penuh (*nS < maxStudent). Memvalidasi format numerik NIS dan memanggil BinarySearchStudent untuk mencegah redundansi.
EditStudent	func EditStudent(scanner *bufio.Scanner, nS *int, nA *int)	Mengubah nama mahasiswa berdasarkan indeks. Melakukan iterasi pada attendanceData untuk memperbarui atribut nama pada semua record yang memiliki NIS serupa.
DeleteStudent	func DeleteStudent(scanner *bufio.Scanner, nS *int, nA *int)	Menghapus record dari studentData . Melakukan pergeseran indeks ke kiri untuk menjaga kerapatan elemen array dan

		menghapus transaksi presensi mahasiswa tersebut di attendanceData .
isNumeric	func isNumeric(s string) bool	Algoritma validasi karakter. ³ Melakukan iterasi byte demi byte pada string untuk memastikan hanya berisi rentang karakter ASCII '0' hingga '9'.
nisLess	func nisLess(a, b string) bool	Melakukan komparasi nilai NIS numerik setelah dikonversi menggunakan strconv.Atoi . Mencegah kegagalan pengurutan pada nilai digit ganjil.
BinarySearchStudent	func BinarySearchStudent(nis string, low, high int) int	Algoritma pencarian biner rekursif/iteratif untuk mendeteksi indeks NIS pada array terurut. Mengembalikan indeks elemen atau -1 jika nihil.
SortStudentAsc	func SortStudentAsc(start int, nS *int)	Mengurutkan array studentData secara menaik menggunakan parameter pembanding dari fungsi nisLess .

```
func isNumeric(s string) bool {
    if len(s) == 0 {
```

```

        return false
    }
    for _, c := range s {
        if c < '0' || c > '9' {
            return false
        }
    }
    return true
}

func nisLess(a, b string) bool {
    ia, ea := strconv.Atoi(a)
    ib, eb := strconv.Atoi(b)
    if ea == nil && eb == nil {
        return ia < ib
    }
    return a < b
}

func AddStudent(scanner *bufio.Scanner, nS *int) {
    if *nS >= maxStudent {
        fmt.Println("\n[!] Error: Student data capacity is full!")
        return
    }

    var inputNIS string
    isValid := false

    for !isValid {
        fmt.Print("Input NIS (Numeric): ")
        if !scanner.Scan() {
            return
        }
        inputNIS = strings.TrimSpace(scanner.Text())

        if isNumeric(inputNIS) {
            SortStudentAsc(1, nS)
            if BinarySearchStudent(inputNIS, 0, *nS-1) != -1 {
                fmt.Println("\n[!] Error: NIS is already registered!")
            } else {
                isValid = true
            }
        } else {
            fmt.Println("\n[!] Error: Invalid NIS format! Must be
numbers.")

```

```

    }
}

fmt.Print("Input Name: ")
if !scanner.Scan() {
    return
}
inputName := strings.TrimSpace(scanner.Text())

if inputName == "" {
    fmt.Println("\n[!] Error: Name cannot be empty!")
    return
}

studentData[*nS].NIS = inputNIS
studentData[*nS].Name = inputName
*nS++

SortStudentAsc(1, nS)
fmt.Println("\n[+] Message: Successfully added student data!")
}

```

3.3. Implementasi Searching

Operasi pencarian merupakan bagian krusial dalam fungsionalitas integritas data sistem presensi ini. Metode pencarian yang dipilih untuk pencarian identitas utama mahasiswa berdasarkan NIS adalah **Pencarian Biner (Binary Search)**. Alasan akademis pemilihan metode ini didasarkan pada efisiensi performa pencarian yang dimilikinya, di mana kompleksitas waktunya dinyatakan dalam notasi Big-O sebagai:

$$O(\log N)$$

Di mana N merepresentasikan jumlah mahasiswa yang terdaftar. Dibandingkan dengan metode Pencarian Sekuensial yang membutuhkan waktu sebesar $O(N)$, pencarian biner memangkas ruang pencarian menjadi setengahnya pada setiap langkah iterasi, menjadikannya sangat cepat saat dihadapkan pada volume data yang maksimal.

Prasyarat mutlak berjalannya algoritma Pencarian Biner adalah data dalam array harus berada dalam kondisi terurut secara menaik. Program ini menjamin prasyarat ini terpenuhi dengan mengeksekusi prosedur *SortStudentAsc* sesaat sebelum pencarian biner dipanggil di dalam fungsi *AddStudent* atau *SearchStudentMenu*.

Mekanisme kerja algoritma *BinarySearchStudent* diimplementasikan dengan membagi rentang indeks pencarian secara dinamis menggunakan variabel pembatas *low* dan *high*. Pada setiap iterasi, algoritma menghitung titik tengah dengan persamaan berikut:

$$mid = low + \frac{high - low}{2}$$

Nilai NIS pada indeks *mid* kemudian dibandingkan dengan target NIS:

- Jika *studentData[mid].NIS == nis*, proses pencarian berhasil dan fungsi langsung mengembalikan nilai indeks *mid*.
- Jika *nisLess(studentData[mid].NIS, nis)* bernilai *true*, hal ini menandakan bahwa target NIS berada di belahan kanan array. Oleh karena itu, batas bawah digeser menjadi *low = mid + 1*.
- Apabila nilai *low* melampaui nilai *high* dan elemen belum ditemukan, pencarian dihentikan dan fungsi mengembalikannya sebagai nilai *-1*.

Penerapan nyata dari pencarian biner ini diintegrasikan ke dalam prosedur pendaftaran mahasiswa baru (*AddStudent*), di mana input NIS divalidasi terlebih dahulu keunikannya. Jika pencarian biner mengembalikan indeks selain *-1*, sistem akan segera menampilkan pesan kesalahan dan menolak proses penyimpanan data baru demi menghindari adanya record ganda.

```
func BinarySearchStudent(nis string, left int, right int) int {
    if left > right {
        return -1
    }
    mid := (left + right) / 2
    if studentData[mid].NIS == nis {
        return mid
    } else if nisLess(nis, studentData[mid].NIS) {
        return BinarySearchStudent(nis, left, mid-1)
    } else {
        return BinarySearchStudent(nis, mid+1, right)
    }
}
```

3.4. Implementasi Sorting

Prosedur pengurutan (*sorting*) memegang peranan ganda di dalam aplikasi ini, yaitu sebagai penyedia prasyarat algoritma pencarian biner dan sebagai penyusun estetika visual pada penyajian laporan statistik. Algoritma pengurutan yang diterapkan didesain secara modular untuk mendukung pengurutan menaik (*ascending*) dan pengurutan menurun (*descending*).

Data utama yang diurutkan meliputi:

1. **Array *studentData* berdasarkan NIS secara menaik:** Urutan ini dijaga secara ketat melalui pemanggilan fungsi *SortStudentAsc*. Masalah klasik pengurutan string (*lexicographical sorting bug*) diselesaikan dengan mengimplementasikan fungsi *nisLess*. Fungsi ini mengekstrak nilai string NIS dan mengubahnya menjadi tipe data numerik integer riil melalui modul *strconv.Atoi* sebelum melakukan operasi perbandingan kurang dari. Tanpa konversi ini, perbandingan biner berbasis kode ASCII akan menyatakan bahwa string "9" lebih besar daripada string "10", yang akan merusak struktur pengurutan numerik NIS dan berakibat pada kegagalan total pencarian biner.
2. **Array *TabStatistics* berdasarkan persentase kehadiran secara menurun:** Pengurutan ini dilakukan di dalam modul pelaporan statistik. Algoritma akan membandingkan atribut *Percentage* dari masing-masing entitas mahasiswa di dalam array hasil perhitungan. Mahasiswa dengan persentase kehadiran terkecil akan ditempatkan di posisi terbawah. Hal ini sangat membantu dosen atau staf administrasi untuk segera mengidentifikasi mahasiswa yang memerlukan pembinaan akademis akibat tingkat absensi yang tinggi.

Skema perbandingan nilai bertipe string NIS yang diimplementasikan di dalam subprogram pembantu *nisLess* dirumuskan sebagai berikut:

$$nisLess(a, b) = \begin{cases} strconv.Atoi(a) < strconv.Atoi(b), & \text{jika konversi berhasil} \\ a < b, & \text{jika konversi gagal (fallback leksikografis)} \end{cases}$$

Dengan mengintegrasikan fungsi tersebut ke dalam loop perbandingan elemen array, kestabilan struktur pengurutan dalam memori utama dapat terjaga secara konsisten.

```
func SortStudentAsc(mode int, nS *int) {
    for i := 0; i < *nS-1; i++ {
        minIdx := i
        for j := i + 1; j < *nS; j++ {
            if mode == 1 {
                if nisLess(studentData[j].NIS, studentData[minIdx].NIS)
            {
                minIdx = j
            }
            } else {
                if strings.ToLower(studentData[j].Name) <
strings.ToLower(studentData[minIdx].Name) {
                    minIdx = j
                }
            }
        }
        studentData[i], studentData[minIdx] = studentData[minIdx],
studentData[i]
    }
}
```

```

func SortStatsDesc(stats *TabStatistics, count int, mode int) {
    for i := 1; i < count; i++ {
        key := stats[i]
        j := i - 1

        switch mode {
        case 1:
            for j >= 0 && nisLess(stats[j].NIS, key.NIS) {
                stats[j+1] = stats[j]
                j--
            }
        case 2:
            for j >= 0 && strings.ToLower(stats[j].Name) <
strings.ToLower(key.Name) {
                stats[j+1] = stats[j]
                j--
            }
        default:
            for j >= 0 && stats[j].Percentage < key.Percentage {
                stats[j+1] = stats[j]
                j--
            }
        }
        stats[j+1] = key
    }
}

```

BAB IV HASIL DAN KESIMPULAN

4.1. Demo Program

Bagian ini menyajikan bukti nyata fungsionalitas program berbasis terminal CLI yang dijalankan menggunakan data dummy hasil inisialisasi awal program.

1. Menu Utama Aplikasi (Main Menu CLI):

Saat pertama kali aplikasi dieksekusi, sistem secara otomatis menjalankan prosedur *SeedDummyData* dan menyajikan antarmuka menu utama yang bersih. Pilihan navigasi disusun menggunakan angka indeks terstruktur.

```
=====
||          STUDENT ATTENDANCE APP          ||
=====
[1] Student Menu
[2] Attendance Menu
[3] Statistics Menu
[0] Exit
=====
Select option:
```

2. Daftar Mahasiswa Terdaftar (Tampilan Data Master):

Ketika pengguna masuk ke Menu Mahasiswa dan memilih opsi "Show Student Data", sistem menampilkan daftar 10 mahasiswa dummy hasil seeding awal yang sudah terurut rapi berdasarkan NIS secara menaik.

```
=====
||          STUDENT MENU          ||
=====
[1] Back to Menu
[2] Add Student
[3] Edit Student
[4] Delete Student
[5] Search Student
[6] Sort Student
[7] Show Student Data
[0] Exit
=====
Select option: 7

=====
| NO | NIS          | NAME                |
=====
| 1  | 103032500146 | Michael Jordan     |
| 2  | 103032500149 | Lionel Messi       |
| 3  | 103032500153 | Cristiano Ronaldo  |
| 4  | 103032500154 | Bill Gate          |
| 5  | 103032500177 | Vladimir Putin     |
| 6  | 103032500180 | Xi Jinping         |
| 7  | 103032540001 | Joe Biden          |
| 8  | 103032540002 | LeBron James       |
| 9  | 103032540004 | Elon Musk           |
| 10 | 103032540005 | Barack Obama       |
=====
```

3. **Demo Validasi Duplikasi Data (Uji Coba Add Student):**

Demonstrasi ini menguji keandalan sistem dalam mencegah redundansi data. Ketika pengguna mencoba mendaftarkan mahasiswa baru dengan NIS "103032540004" yang sudah dimiliki oleh mahasiswa bernama Elon Musk, algoritma *BinarySearchStudent* langsung mendeteksi duplikasi tersebut. Sistem menghentikan pendaftaran dan menampilkan indikasi galat.

```
=====
||                               ||
||                               ||
=====
[1] Back to Menu
[2] Add Student
[3] Edit Student
[4] Delete Student
[5] Search Student
[6] Sort Student
[7] Show Student Data
[0] Exit
=====
Select option: 2
Input NIS (Numeric): 103032540004

[!] Error: NIS is already registered!
Input NIS (Numeric): |
```

4. Laporan Statistik Kehadiran (Statistics Report):

Menampilkan hasil kalkulasi kehadiran mahasiswa secara real-time berdasarkan 3 hari sesi pertemuan kelas historis yang ada di dalam database. Laporan ini menampilkan detail total pertemuan, jumlah kehadiran mahasiswa, serta kalkulasi persentase yang diurutkan menurun.

```

=====
||          STATISTICS MENU          ||
=====
[1] Back to Menu
[2] Show Student Statistics
[3] Search Student Statistic
[4] Sort Student Statistic
[0] Exit
=====
Select option: 2
=====
| NIS          | NAME          | PRS/MTG | PERCENT |
=====
| 103032500146 | Michael Jordan | 3 / 3   | 100.00% |
| 103032500153 | Cristiano Ronaldo | 3 / 3   | 100.00% |
| 103032500177 | Vladimir Putin | 3 / 3   | 100.00% |
| 103032500180 | Xi Jinping     | 3 / 3   | 100.00% |
| 103032540002 | LeBron James  | 3 / 3   | 100.00% |
| 103032540005 | Barack Obama  | 3 / 3   | 100.00% |
| 103032500149 | Lionel Messi   | 2 / 3   | 66.67 % |
| 103032540004 | Elon Musk     | 2 / 3   | 66.67 % |
| 103032500154 | Bill Gate     | 1 / 3   | 33.33 % |
| 103032540001 | Joe Biden     | 0 / 3   | 0.00 % |
=====

```

4.2. Kesimpulan

Berdasarkan seluruh proses rancang bangun, implementasi, serta pengujian yang telah dilaksanakan pada Aplikasi Absensi Siswa berbasis bahasa pemrograman Go ini, dapat ditarik beberapa kesimpulan akademis yang komprehensif:

1. **Keberhasilan Modularitas dan Fungsionalitas:** Seluruh modul program yang direncanakan—mulai dari manajemen data master mahasiswa (CRUD), perekaman transaksi presensi harian, hingga kalkulasi persentase kehadiran—telah berhasil diimplementasikan secara utuh dan terbebas dari kesalahan runtime. Pemanfaatan pointer dalam memanipulasi parameter pencatat jumlah data terbukti efektif menjaga konsistensi state dalam memori tanpa kebocoran data.
2. **Kekuatan Integrasi Algoritma:** Penerapan algoritma pengurutan yang dikombinasikan dengan fungsi perbandingan numerik presisi tinggi (*nisLess*) terbukti sukses mengeliminasi kelemahan klasik sorting string. Hal ini memberikan jaminan keandalan mutlak bagi algoritma Pencarian Biner (*Binary Search*) dalam memvalidasi keunikan NIS secara instan dengan efisiensi waktu $O(\log N)$, sehingga integritas data master terbebas dari duplikasi.

3. **Efisiensi Struktur Memori Statis:** Penggunaan tipe data array statis dengan ukuran tetap (*TabStudent* dan *TabAttendance*) memberikan pemahaman mendalam mengenai batasan komputasi fisik dan alokasi memori deterministik. Rekayasa relasi data antar-array menggunakan kunci unik NIS mampu menyimulasikan sistem basis data relasional mini yang efisien dan cepat di dalam RAM, menjadikannya model pembelajaran yang solid bagi penerapan konsep-konsep Algoritma Pemrograman.